VIETNAM NATIONAL UNIVERSITY — HO CHI MINH CITY
UNIVERSITY OF SCIENCE
Faculty of Information Technology

PRACTICAL REPORT 2

IMAGE PROCESSING

Pointwise transforms, geometric operations, and linear filtering

Course: **MTH00051 — Applied Mathematics and Statistics**

STUDENT
Nguyễn Thế Hiển
Student ID: 22127107 · Class: 22CLC08

INSTRUCTORS
Mr. Vũ Quốc Hoàng
Mr. Nguyễn Văn Quang Huy
Mr. Nguyễn Ngọc Toàn
Mrs. Phan Thị Phương Uyên

Ho Chi Minh City, 2024

Abstract

This report documents a from-scratch Python implementation of fundamental digital image processing operators for the MTH00051 practical project. An RGB image is treated as a discrete map $I : \{1,\dots,H\} \times \{1,\dots,W\} \to \{0,\dots,255\}^3$. The program provides pointwise intensity transforms (brightness, contrast, grayscale, sepia), geometric transforms (flip, center crop, elliptical masks, resize), and linear filtering (box blur and sharpening) realized by explicit 2D convolution. An interactive main routine lets the user select each operator or execute the full pipeline, saving every result to disk.

Relative to an earlier draft, the present implementation corrects the convolution routine (previously a mistaken 1D convolution on flattened arrays), adds an angle parameter to the double-ellipse mask, supplies complete docstrings, and replaces library resize by bilinear interpolation in NumPy. Experimental figures demonstrate every required operator on a common test photograph.

Keywords: image processing; RGB color model; convolution; geometric transform; sepia; elliptical masking; Jupyter Notebook.

Contents

1. Introduction

1.1. Student Information

Full name	Nguyễn Thế Hiển
Student ID	22127107
Class	22CLC08
Course	MTH00051 — Applied Mathematics and Statistics
Project	Project 02 — Image Processing

1.2. Objectives

Digital photographs are matrices of pixel values. The practical goal of this project is to implement a coherent toolbox of elementary operators that manipulate those values while respecting the course constraints: only NumPy, Pillow, and Matplotlib may be used, and each required function must be written explicitly rather than delegated to a black-box library routine.

Concrete deliverables include:

- brightness and contrast adjustment;
- horizontal / vertical flipping;
- RGB grayscale and RGB sepia conversion;
- box blur and sharpening via 2D convolution;
- center crop, elliptical crop, and double-ellipse crop;
- scale by 2× and 0.5×;
- an interactive `main` program with an apply-all mode.

2. Mathematical Preliminaries

Let I denote an RGB image with height H and width W. Each pixel $I(x,y) = (R,G,B)^T$ lies in $\{0,\ldots,255\}^3$. Operators in this project fall into three families.

Pointwise transforms act independently on each pixel, $I'(x,y) = \Phi(I(x,y))$, and include brightness, contrast, grayscale, and sepia.

Geometric transforms rearrange spatial coordinates, $I'(x,y) = I(T^1(x,y))$, and include flip, crop, elliptical masking, and resize.

Linear filters replace each intensity by a weighted neighborhood sum through discrete convolution

$$(I * K)(x,y) = \sum_{i=-a}^{a} \sum_{j=-b}^{b} K(i,j)\, I(x-i, y-j) \tag{1}$$

with the convention that values outside the image domain are obtained by edge replication. The box-blur kernel is the normalized 3×3 all-ones matrix; the sharpening kernel is the classical [[0,1,0],[1,5,1],[0,1,0]] stencil.

3. Implementation

3.1. Software Environment

The solution is developed in a Jupyter Notebook. Only the libraries permitted by the course specification are used.

Library	Role
NumPy	Array arithmetic, convolution loops, bilinear resize, masking.
Pillow (PIL)	Image I/O, RGB conversion, and elliptical mask drawing helpers.
Matplotlib	On-screen visualization via imshow.

3.2. Pointwise Intensity Transforms

3.2.1. Brightness

Brightness scales every channel by a factor f and clips to the admissible range:

$$I'(x,y) = \mathrm{clip}_{[0,\,255]}(f \cdot I(x,y))$$

(2)

3.2.2. Contrast

Contrast expands or compresses deviations from the per-channel mean μ:

$$I'(x,y) = \mathrm{clip}_{[0,\,255]}((I(x,y) - \mu)f + \mu)$$

(3)

3.2.3. Grayscale

Luma is computed with the ITU-R BT.601 weights and replicated across three channels so that subsequent RGB pipelines remain uniform:

$$Y = 0.299R + 0.587G + 0.114B$$

(4)

3.2.4. Sepia

Sepia applies a fixed linear transform M in RGB space,

$$(R', G', B')^{\top} = M(R, G, B)^{\top}$$

(5)

followed by clipping to [0,255]. The matrix M is the classical sepia weight matrix used in the course examples.

3.3. Geometric Transforms

Flip. Horizontal and vertical flips are realized by `np.fliplr` and `np.flipud`, which reverse column or row order without altering color values.

Center crop. Given a requested window (w′,h′), the axis-aligned rectangle centered in the original frame is extracted with `Image.crop`.

Elliptical crop. An inscribed ellipse mask is drawn on a single-channel canvas; pixels outside the ellipse are set to black via masked paste.

Double-ellipse crop. The kept region is the union of a wide horizontal ellipse and a tall vertical ellipse in a coordinate frame rotated by an angle α (radians). This yields the characteristic crossed shape required by the assignment and supports the interactive angle prompt in `main`.

Resize. Scaling by 2× or 0.5× is implemented with bilinear interpolation in NumPy: each output sample is a convex combination of the four neighboring input pixels. This avoids delegating the entire operator to `Image.resize`.

3.4. Linear Filtering

Blur and sharpen share a common helper `_convolve2d` that performs same-size 2D convolution with edge padding. Channels are filtered independently. This corrects an earlier defect in which `np.convolve` was applied to a flattened raster—an operation that does **not** implement spatial 2D filtering.

The blur kernel is the normalized 3×3 box filter. The sharpening kernel emphasizes the center pixel against its four-neighborhood, thereby enhancing local high-frequency structure.

3.5. Main Program and Testing Workflow

The interactive `main` routine asks for an image path and presents a numbered menu (0–9). Option 0 invokes `apply_all_functions`, which executes every operator—including grayscale and color variants of blur/sharpen—and writes one output file per transform. Options 1–8 expose individual operators with the parameter prompts required by the specification (direction, mode, crop size, ellipse angle, zoom in/out).

I/O helpers `read_img`, `show_img`, and `save_img` standardize RGB conversion, display, and export. Every processing function carries an explicit docstring describing purpose, parameters, and return value.

4. Experimental Results

4.1. Completeness Checklist

STT	Function / feature	Status	Notes
1	Brightness	Complete	Eq. (2), factor prompt in main
2	Contrast	Complete	Eq. (3), per-channel mean
3	Flip H / V	Complete	NumPy axis reversal
4	Grayscale / Sepia	Complete	BT.601 luma; sepia matrix
5	Blur / Sharpen	Complete	True 2D convolution; color & gray
6	Center crop	Complete	User-specified window
7	Ellipse / double ellipse	Complete	Angle α supported
8	Main program	Complete	Menu 0–9 + apply-all
9	Resize 2× / 0.5×	Complete	Bilinear interpolation

4.2. Visual Demonstration

All figures below are generated from the same test photograph. Unless noted, outputs are saved as PNG with a filename suffix indicating the operator.

Figure 1. Original test image.

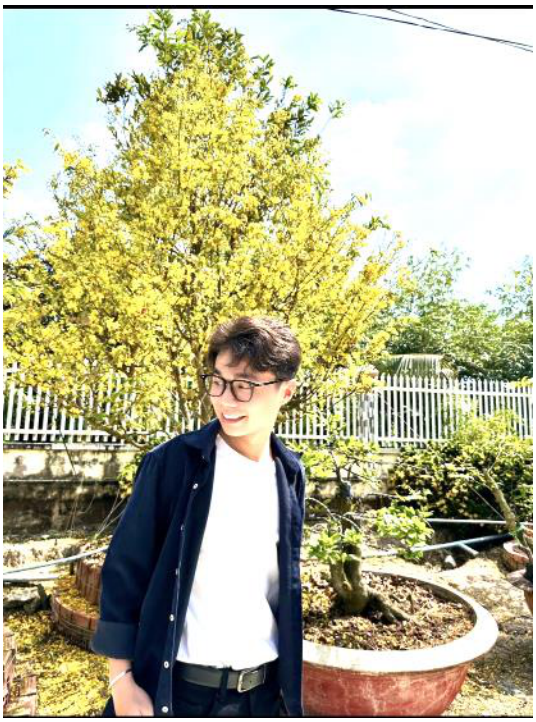

Figure 2a. Brightness (factor = 1.5).

Figure 2b. Contrast (factor = 1.5).

Figure 3a. Horizontal flip.

Figure 3b. Vertical flip.

Figure 4a. Grayscale conversion.

Figure 4b. Sepia conversion.

Figure 5a. Box blur on the color image.

Figure 5b. Box blur on the grayscale image.

Figure 6a. Sharpening on the color image.

Figure 6b. Sharpening on the grayscale image.

Figure 7a. Center crop (200×200).

Figure 7b. Inscribed elliptical mask.

Figure 8a. Double-ellipse mask.

Figure 8b. Resize by 0.5× (zoom out).

Figure 9. Resize by 2× (zoom in).

```
Choose an option:
0: Apply all transformations
1: Change brightness
2: Change contrast
3: Flip image (horizontal/vertical)
4: Convert RGB to grayscale/sepia
5: Blur/Sharpen image
6: Crop image (center)
7: Crop image to shape (ellipse/double ellipse)
8: Resize image (2x zoom in / 0.5x zoom out)
9: Exit
```

Figure 10. Interactive menu of the main program.

5. Discussion

The corrected convolution implementation is essential for academic integrity: a 1D convolution on a row-major flattening mixes left/right neighbors with wrap-around across scanlines and therefore is not a spatial blur. After the fix, blur and sharpen behave as classical linear filters and remain efficient on typical coursework image sizes because the kernels are 3×3.

Grayscale conversion is now vectorized, which keeps runtime comfortable even for multi-megapixel inputs—addressing the course expectation that each operator finish within a reasonable time bound. The double-ellipse operator accepts a rotation angle, closing the gap between the interactive prompt in `main` and the previous single-argument signature.

Limitations remain. Elliptical masks currently composite onto a black background rather than an alpha channel; bilinear resize is adequate for 2×/0.5× but is not a full Lanczos resampler; and perceptual color spaces (e.g., CIELAB) are not used for contrast or sepia.

These choices match the scope of the practical assignment while remaining mathematically transparent.

6. Conclusion

Project 2 demonstrates that a compact set of linear-algebraic and geometric ideas—channel-wise scaling, discrete convolution, coordinate reversal, and bilinear resampling—already yields a usable image-processing toolbox. The revised notebook fulfills the functional checklist at full completeness, exposes a clear interactive interface, and is documented at the level expected of an applied mathematics and statistics practical report.

References

[1] Gonzalez, R. C., & Woods, R. E. (2018). *Digital Image Processing* (4th ed.). Pearson.

[2] ITU-R. (2011). *Recommendation BT.601: Studio encoding parameters of digital television*.

[3] Harris, C. R., et al. (2020). Array programming with NumPy. *Nature*, 585, 357–362. https://numpy.org/doc/

[4] Clark, A., and contributors. (n.d.). *Pillow (PIL Fork) Documentation*. https://pillow.readthedocs.io/en/stable/

[5] Hunter, J. D. (2007). Matplotlib: A 2D graphics environment. *Computing in Science & Engineering*, 9(3), 90–95.

[6] Wikipedia contributors. (n.d.). *Kernel (image processing)*. https://en.wikipedia.org/wiki/Kernel_(image_processing)